МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

Розрахунково-графічна робота

На тему: «Принципи адресації за вмістом замість IP»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

Зміст

1. Вступ.....3
2. Теоретичний розділ.....6
2.1 Загальна характеристика адресації даних.....6
2.2 Огляд основних понять і підходів.....7
2.3 Класична IP адресація та її обмеження.....8
2.4 Принцип адресації за вмістом.....9
2.5 Хешування.....11
2.6 DHT, Merkle структури та організація пошуку даних.....12
2.7 Дедуплікація, реплікація та використання у розподілених сховищах...13
3. Практичний розділ.....15
3.1 Ідея практичної реалізації теми.....15
3.2. Створення тестових даних та підготовка середовища.....15
3.3. Обчислення SHA 256 хешів для файлів.....17
3.4. Розбиття файлів на блоки та створення сховища за хешем.....18
3.5. Результати роботи програми та пояснення отриманих даних.....19
Висновок.....22
Додаток А.....23
Список використаних джерел.....24

1. Вступ

У сучасних інформаційних системах обсяг даних постійно зростає, а способи їх зберігання, передавання та пошуку стають дедалі складнішими. Якщо раніше більшість інформаційних ресурсів зберігалася на конкретних серверах, до яких користувач звертався за певною мережевою адресою, то сьогодні дані часто розміщуються у розподілених середовищах, хмарних сховищах, однорангових мережах та великих системах обробки даних. У таких умовах важливим стає не тільки питання швидкого доступу до інформації, а й питання її цілісності, доступності, надійного пошуку та захисту від втрати або підміни.

Класичний підхід до доступу до ресурсів у мережі ґрунтується на IP адресації. У цьому випадку користувач або програмна система звертається до конкретного вузла мережі, тобто до певного місця, де розміщено потрібні дані. Такий підхід є зрозумілим і широко використовується в традиційних клієнт серверних системах. Наприклад, вебсайт, база даних або файловий сервер мають адресу, за якою до них можна звернутися. Однак у розподілених системах такий підхід має певні обмеження. Якщо сервер недоступний, перевантажений, змінив адресу або був вилучений із мережі, доступ до даних може бути втрачений навіть у тому випадку, якщо копії цих даних фізично існують на інших вузлах.

Альтернативним підходом є адресація за вмістом. Її основна ідея полягає в тому, що дані ідентифікуються не за місцем свого розташування, а за власним вмістом. Для цього використовуються хеш функції, які формують унікальний цифровий відбиток файлу або окремого блоку даних. Якщо вміст даних залишається однаковим, то й хеш буде однаковим. Якщо змінити хоча б один символ або один байт, хеш повністю зміниться. Завдяки цьому хеш може використовуватися як ідентифікатор об’єкта, а також як засіб перевірки його цілісності.

Актуальність теми адресації за вмістом особливо зростає у контексті технологій підтримки розподілених баз і сховищ даних. У таких системах дані

можуть дублюватися на кількох вузлах, переноситися між серверами, розбиватися на блоки, реплікуватися та відновлюватися після відмов. У цьому випадку важливо мати механізм, який дозволяє точно визначити, які саме дані потрібні користувачу або системі, незалежно від того, на якому вузлі вони зберігаються. Адресація за вмістом дозволяє вирішувати це завдання, оскільки звернення відбувається не до конкретного сервера, а до конкретного вмісту. Тема є важливою, оскільки вона пов’язана з принципами організації розподіленого зберігання, пошуку даних, реплікації, дедуплікації та забезпечення цілісності інформації. У розподілених базах даних і сховищах часто виникає проблема дублювання однакових фрагментів даних. Якщо система може визначати об’єкти за їхнім хешем, вона може не зберігати однакові блоки повторно. Це дозволяє зменшити обсяг використаної пам’яті та підвищити ефективність зберігання. Крім того, адресація за вмістом дає можливість перевірити, чи не було пошкоджено або змінено дані під час передавання або зберігання.

Метою роботи є дослідження принципів адресації за вмістом замість класичної IP адресації та демонстрація можливості застосування цього підходу в розподілених системах зберігання даних. У межах роботи необхідно показати, як дані можуть ідентифікуватися за допомогою хеш функцій, як такий підхід відрізняється від звернення до конкретної IP адреси, а також які переваги він дає для зберігання, пошуку, перевірки цілісності та усунення дублювання даних. Для досягнення поставленої мети в роботі передбачається виконати кілька основних завдань. Насамперед необхідно розглянути загальні принципи IP адресації та пояснити, чому цей підхід орієнтований на місце розташування ресурсу. Далі потрібно розкрити поняття адресації за вмістом і пояснити, яким чином хеш може виступати ідентифікатором даних. Також важливо охарактеризувати роль хеш функцій, зокрема алгоритму SHA 256, у формуванні цифрового відбитка інформаційного об’єкта. Окрему увагу слід приділити поняттям CID, Merkle структур і розподілених хеш таблиць,

оскільки вони є важливими елементами сучасних систем, що використовують content addressing.

Ще одним завданням роботи є виконання практичного моделювання спрощеного сховища з адресацією за вмістом. У такій моделі дані можуть бути розбиті на окремі блоки, для кожного блоку може бути обчислений хеш, а самі блоки можуть зберігатися за цим хешем. Якщо два блоки мають однаковий вміст, вони отримують однаковий ідентифікатор, тому немає потреби зберігати їх повторно. Це дозволяє продемонструвати принцип дедуплікації та виконати розрахунок економії пам’яті. Також у межах роботи доцільно показати, як змінюється хеш при зміні вмісту, і пояснити, чому це важливо для перевірки цілісності даних.

Об’єктом дослідження у даній роботі є процес адресації, зберігання та пошуку даних у розподілених інформаційних системах. Саме ці процеси є важливими для побудови надійних систем обробки та зберігання великих обсягів інформації. У межах об’єкта дослідження розглядається, як користувач або програмна система може знайти потрібний ресурс, як забезпечується доступ до даних, а також як можна перевірити правильність отриманої інформації. Предметом дослідження є принципи адресації за вмістом, хешування даних, формування ідентифікаторів вмісту, дедуплікація, перевірка цілісності та використання таких підходів у розподілених сховищах. Особливу увагу приділено тому, як хеш функції дозволяють замінити залежність від конкретної IP адреси на залежність від самого вмісту даних. Це дає змогу краще зрозуміти різницю між традиційною адресацією за місцем розташування та сучасною адресацією за вмістом.

2. Теоретичний розділ

2.1 Загальна характеристика адресації даних

Адресація даних є одним із базових принципів роботи комп’ютерних мереж, баз даних і розподілених систем зберігання інформації. За допомогою адресації система визначає, як знайти потрібний ресурс, як звернутися до нього і яким способом отримати дані. У традиційних інформаційних системах найчастіше використовується підхід, за якого ресурс пов’язується з певним місцем розташування. Наприклад, файл, вебсторінка або база даних можуть бути доступні через адресу сервера, на якому вони розміщені.

Класичним прикладом такого підходу є IP адресація. У цьому випадку кожен вузол у мережі має власну IP адресу, яка дозволяє іншим пристроям звертатися до нього. Така модель добре працює у звичайних клієнт серверних системах, де користувач або програма надсилає запит до конкретного сервера, а сервер повертає потрібну інформацію.

Однак у сучасних розподілених системах дані не завжди зберігаються в одному місці. Один і той самий об’єкт може мати кілька копій, розміщених на різних вузлах. Дані можуть реплікуватися, переноситися, кешуватися або відновлюватися після відмов. У таких умовах важливим стає не тільки питання місця зберігання, а й питання точної ідентифікації самих даних.

Саме тому виникає підхід адресації за вмістом. Він дозволяє звертатися не до конкретного сервера, а до конкретного інформаційного об’єкта. У цьому випадку основою ідентифікації є не IP адреса, а хеш вмісту. Такий підхід є важливим для розподілених сховищ, систем резервного копіювання, одноранговых мереж і технологій, де потрібно забезпечити цілісність, доступність і відсутність дублювання даних.

2.2 Огляд основних понять і підходів

У межах теми адресації за вмістом розглядається кілька ключових понять, які пов’язані між собою. Насамперед це класична IP адресація, яка використовується для ідентифікації вузлів у мережі. Вона відповідає на питання, де знаходиться ресурс. Такий підхід називають адресацією за місцем розташування, оскільки користувач або програмна система звертається до конкретного сервера, пристрою або вузла мережі.

Другим важливим поняттям є адресація за вмістом. Вона відрізняється тим, що ідентифікує не місце зберігання, а самі дані. Для цього застосовуються хеш функції. Вони формують цифровий відбиток файлу або блоку даних, який може використовуватися як ідентифікатор. Якщо вміст залишається однаковим, то й хеш також залишається однаковим. Якщо ж змінити навіть невелику частину даних, хеш зміниться.

Поняття	**Коротке пояснення**
IP адресація	спосіб звернення до конкретного вузла або сервера в мережі
Адресація за вмістом	спосіб ідентифікації даних за їхнім вмістом
Хеш	цифровий відбиток даних
Хеш функція	алгоритм, який перетворює дані на хеш фіксованої довжини
SHA 256	поширений алгоритм хешування
CID	ідентифікатор вмісту, який використовується для звернення до даних
DHT	розподілена хеш таблиця для пошуку даних у мережі
Merkle структура	структура, яка дозволяє перевіряти цілісність набору блоків
Дедуплікація	усунення повторюваних блоків даних
Реплікація	створення копій даних на кількох вузлах

Табл.1 Ключові визначення

2.3 Класична IP адресація та її обмеження

Класична IP адресація є основою роботи більшості комп’ютерних мереж. IP адреса використовується для ідентифікації пристрою або сервера в мережі. Коли користувач відкриває сайт, підключається до бази даних або завантажує файл, його запит надсилається до конкретного вузла, який має певну IP адресу.

У такій моделі ресурс прив’язаний до місця свого розташування. Наприклад, якщо файл зберігається на сервері, користувач повинен звернутися саме до цього сервера. Якщо сервер доступний, користувач отримує потрібні дані. Якщо сервер недоступний, перевантажений або змінив адресу, доступ доресурсу може бути ускладнений або повністю втрачений.

Головну ідею IP адресації можна сформулювати так: користувач звертається не безпосередньо до самих даних, а до місця, де ці дані знаходяться. Тобто IP адреса відповідає на питання про місце розташування ресурсу.

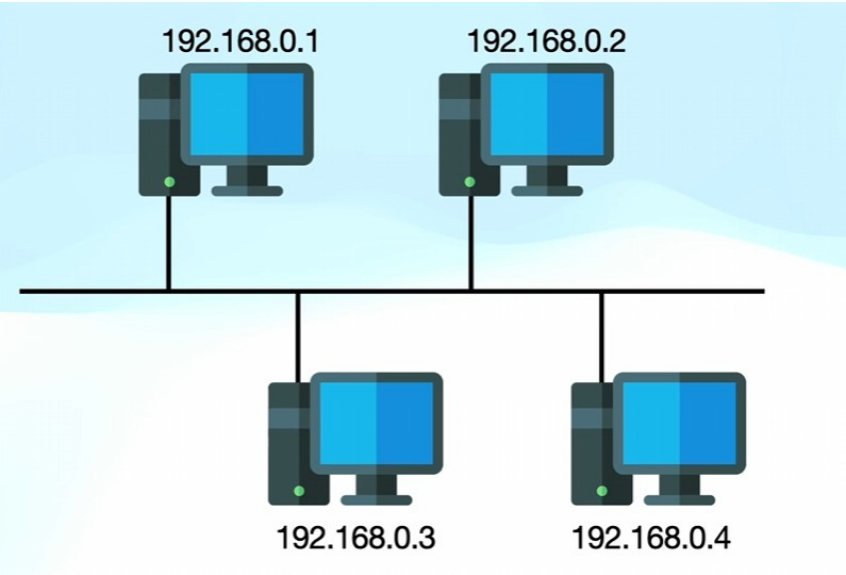


Рис. 1 IP - адресація

Користувач → IP адреса сервера → файл на сервері

Основні обмеження класичної IP адресації:

1. Залежність від конкретного сервера. Якщо дані прив’язані до певного сервера, то доступ до них залежить від доступності цього сервера. У разі відмови вузла користувач може не отримати потрібний ресурс.
2. Відсутність прямої перевірки цілісності даних. IP адреса показує, куди потрібно звернутися, але вона не гарантує, що отриманий файл не був змінений або пошкоджений.
3. Проблема дублювання даних. Однакові файли можуть зберігатися на різних серверах, але система не завжди може автоматично визначити, що це один і той самий вміст.
4. Складність роботи з розподіленими сховищами. У розподілених системах дані можуть зберігатися на багатьох вузлах одночасно, тому прив’язка тільки до IP адреси не завжди є достатньою.
5. Залежність від мережевої адреси. Якщо ресурс переміщується на інший сервер, його адреса може змінитися. Це створює додаткові складнощі для доступу до даних.

2.4 Принцип адресації за вмістом

Адресація за вмістом є підходом, за якого дані ідентифікуються на основі власного вмісту. У такій моделі головним є не те, на якому сервері знаходиться файл, а те, який саме вміст має цей файл. Для цього використовується хешування.

Коли файл або блок даних надходить в систему, для нього обчислюється хеш. Отриманий хеш використовується як ідентифікатор об’єкта. Якщо користувачу потрібен певний файл, система може шукати його за хешем, а не за IP адресою конкретного сервера.

По суті принцип адресації за вмістом можна подати наступним чином : дані → хеш функція → хеш → адреса вмісту

У класичній IP моделі запит будується навколо питання: де знаходиться файл. В адресації за вмістом запит будується навколо іншого питання: який

саме файл потрібен. Це означає, що адресація за вмістом відповідає не на питання місця, а на питання ідентичності даних.

Наприклад, якщо один і той самий файл зберігається на кількох вузлах, його хеш буде однаковим. Отже, система може отримати цей файл з будь якого доступного вузла, який має відповідний вміст. При цьому не настільки важливо, де саме фізично зберігається файл, бо головним ідентифікатором є його хеш.

Послідовність роботи адресації за вмістом можна описати так:

1. У систему надходить файл або блок даних.
2. Для цього об’єкта обчислюється хеш.
3. Хеш використовується як ідентифікатор або адреса об’єкта.
4. Об’єкт зберігається у сховищі за цим хешем.
5. Під час запиту користувач або програма звертається до хешу.
6. Система знаходить об’єкт із відповідним хешем.
7. Після отримання даних можна повторно обчислити хеш і перевірити цілісність.

Головна особливість такого підходу полягає в тому, що адреса залежить від самого вмісту. Якщо вміст не змінюється, адреса залишається однаковою. Якщо вміст змінюється, змінюється і хеш, тобто змінюється адреса. Для розподілених систем це має велике значення. Дані можуть зберігатися на різних вузлах, але якщо вони мають однаковий вміст, вони матимуть однаковий ідентифікатор. Це дозволяє зменшити залежність від конкретного сервера, перевіряти цілісність даних і виявляти дублікати.

2.5 Хешування

Хешування є основою адресації за вмістом. Хеш функція перетворює вхідні дані довільного розміру на значення фіксованої довжини. Це значення називається хешем або цифровим відбитком даних.

Наприклад, файл може мати розмір кілька кілобайтів або кілька мегабайтів, але результат хешування алгоритмом SHA 256 завжди матиме фіксовану довжину. Це дозволяє використовувати хеш як компактний ідентифікатор вмісту.

SHA 256 є криптографічною хеш функцією, яка належить до сімейства алгоритмів SHA 2. Її призначення полягає у перетворенні вхідних даних довільного розміру на хеш фіксованої довжини. Результат роботи SHA 256 завжди має довжину 256 біт, тобто 32 байти. У шістнадцятковому вигляді такий хеш зазвичай записується як рядок із 64 символів. У спрощеному вигляді роботу алгоритму можна подати формулою:

,

де M це вхідне повідомлення, файл або блок даних, а H це отриманий 256 бітний хеш

Основна властивість SHA 256 полягає в тому, що однакові вхідні дані завжди формують однаковий хеш. Якщо ж змінити хоча б один символ або один байт у початкових даних, результат хешування буде іншим. Завдяки цьому SHA 256 може використовуватися як цифровий відбиток даних.

Для адресації за вмістом SHA 256 є важливим тому, що хеш може виконувати роль ідентифікатора інформаційного об’єкта. У такому випадку адреса залежить не від місця зберігання файлу, а від його вмісту. Якщо два файли або блоки мають однаковий вміст, вони матимуть однаковий хеш. Якщо вміст змінюється, змінюється і хеш, тобто змінюється адреса вмісту.

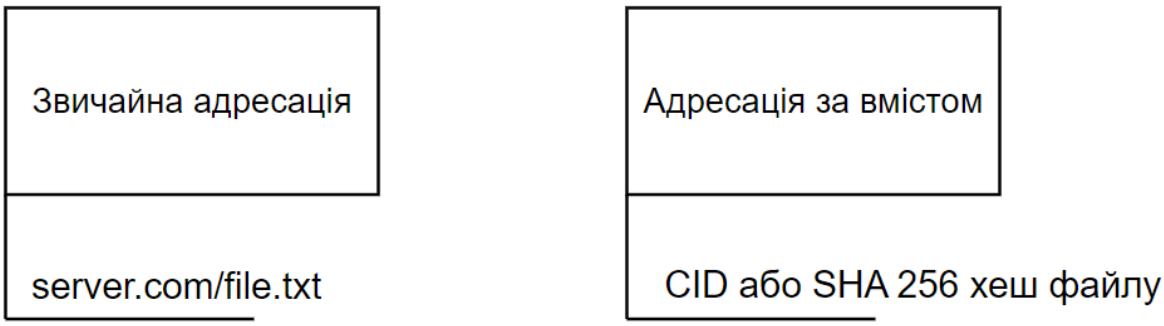


Рис. 2 Умовне порівняння адресацій

2.6 DHT, Merkle структури та організація пошуку даних

Адресація за вмістом дозволяє отримати унікальний ідентифікатор даних у вигляді хешу або CID. Але після цього виникає інше питання: якщо система знає хеш потрібного об’єкта, то як знайти вузол, на якому цей об’єкт зберігається. Сам по собі хеш показує, які саме дані потрібні, але не показує їхнє фізичне місце зберігання. Для вирішення цієї задачі у розподілених системах використовуються спеціальні механізми пошуку, зокрема DHT.

DHT (Distributed Hash Table), тобто розподілена хеш таблиця. Її можна уявити як каталог, який зберігається не на одному центральному сервері, а розподіляється між багатьма вузлами мережі. У такій таблиці ключем є хеш або CID, а значенням є інформація про вузли, які зберігають відповідний об’єкт.

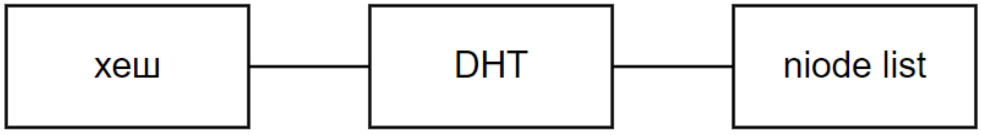


Рис.3 Спрощений DHT

Хеш або CID	Вузли, які мають дані

H1	Вузол 1, Вузол 3
H2	Вузол 2
H3	Вузол 1, Вузол 2, Вузол 4

Табл. 2 DHT

Якщо користувачу потрібен об’єкт із хешем H1, система звертається до DHT і отримує інформацію, що цей об’єкт зберігається на Вузлі 1 та Вузлі 3. Після цього дані можуть бути отримані з одного з доступних вузлів. Таким чином, DHT виконує роль механізму пошуку в системах з адресацією за вмістом.

Важливо розуміти, що DHT не змінює сам принцип адресації за вмістом. Вона лише допомагає знайти, де саме у розподіленій мережі є потрібний об’єкт. Хеш або CID відповідає на питання, які дані потрібні, а DHT допомагає відповісти на питання, на яких вузлах ці дані можна отримати.

Ще одним важливим поняттям є Merkle структура. Вона використовується для організації великих об’єктів, які складаються з багатьох блоків. Замість того щоб працювати з великим файлом як з одним цілим об’єктом, система може розбити його на частини. Для кожного блоку обчислюється власний хеш, а потім із цих хешів формується загальний кореневий хеш.

2.7 Дедуплікація, реплікація та використання у розподілених сховищах

Адресація за вмістом має важливе практичне значення для розподілених сховищ даних. Вона дозволяє не тільки ідентифікувати об’єкти за хешем, але й ефективніше організовувати зберігання, перевірку та копіювання даних між вузлами. Найбільш помітно це проявляється у процесах дедуплікації та реплікації.

Дедуплікація це процес виявлення та усунення повторюваних даних. У звичайному сховищі однакові файли або однакові частини файлів можуть зберігатися багато разів. Це призводить до зайвого використання пам’яті. У системі з адресацією за вмістом така проблема зменшується, тому що однакові дані мають однаковий хеш.

Наприклад, якщо два блоки даних мають однаковий вміст, то після обчислення SHA 256 вони отримають однаковий хеш. Система може зробити висновок, що це один і той самий блок, тому зберігати його повторно не потрібно. Замість цього можна зберегти один фізичний блок, а інші файли будуть посилатися на нього.

Блок	**Вміст**	**Хеш**	**Дія системи**
B1	Текст A	H1	зберегти
B2	Текст B	H2	зберегти
B3	Текст A	H1	не зберігати повторно
B4	Текст C	H3	зберегти

Табл.3 Система дедуплікації

У цьому прикладі блоки B1 і B3 мають однаковий вміст, тому вони мають однаковий хеш H1. Система зберігає блок лише один раз. Це дозволяє зменшити обсяг використаної пам’яті та уникнути зберігання зайвих копій.
Дедуплікація особливо корисна у таких випадках:

1. резервне копіювання даних;
2. файлові сховища;
3. хмарні сховища;
4. архіви;
5. розподілені бази даних;
6. системи, де багато схожих або повторюваних файлів.

В системах з адресацією за вмістом реплікація поєднується з хешуванням. Якщо один і той самий блок зберігається на кількох вузлах, його хеш має бути однаковим на всіх копіях. Це дозволяє перевірити, що копії справді містять однакові дані.

3. Практичний розділ

3.1 Ідея практичної реалізації теми

У практичному розділі було реалізовано спрощену модель сховища з адресацією за вмістом. Основна мета практичної частини полягає в тому, щоб показати, як дані можуть ідентифікуватися не за місцем зберігання, а за власним вмістом. У класичному підході файл зазвичай знаходиться за шляхом або за адресою сервера. У запропонованій моделі головним ідентифікатором є хеш, який обчислюється на основі вмісту файлу або блоку даних.

У практичній частині створюються тестові файли, частина яких має однакові або повторювані фрагменти. Потім файли розбиваються на блоки. Для кожного блоку обчислюється SHA 256 хеш. Якщо блок із таким хешем уже був збережений раніше, він позначається як дублікат і повторно не записується у сховище. Таким чином демонструється дедуплікація даних.

У цій частині не реалізується повноцінна мережева система або реальний IPFS. Натомість створюється навчальна модель, яка показує головний принцип теми: адресою даних виступає не IP адреса сервера, а хеш самого вмісту.

3.2. Створення тестових даних та підготовка середовища

На першому етапі було імпортовано необхідні бібліотеки Python. Бібліотека hashlib використовується для обчислення SHA 256 хешів. Бібліотека os потрібна для створення папок і роботи з файлами. Бібліотека pandas використовується для формування таблиць результатів. Бібліотека matplotlib. pyplot потрібна для побудови графіків.

```
import hashlib
import os
import pandas as pd
import matplotlib.pyplot as plt
```

Рис. 4 Підключення бібліотек

Далі створюється папка test_files, у якій будуть розміщені тестові файли. У практичній частині створено чотири текстові файли. Для демонстрації дедуплікації в них спеціально використано повторювані фрагменти. Зокрема, змінна common_text містить спільний текст, який повторюється у кількох файлах.

```
os.makedirs("test_files", exist_ok=True)

common_text = (
    "Це спільний фрагмент даних. "
    "Він повторюється у кількох файлах і потрібен для демонстрації дедуплікації. "
    "У системі з адресацією за вмістом однакові блоки мають однаковий хеш. "
)


files = {
    "file1.txt": common_text + unique_text_1 + common_text,
    "file2.txt": common_text + unique_text_2 + common_text,
    "file3.txt": unique_text_3 + common_text + unique_text_1,
    "file4.txt": common_text + unique_text_1 + common_text
}
```

Рис.5 Псевдокод створення файлів

Цей етап потрібен для того, щоб підготувати дані для моделювання. Оскільки частина вмісту повторюється, надалі можна буде показати, як адресація за вмістом знаходить однакові блоки за однаковими хешами.

3.3. Обчислення SHA 256 хешів для файлів

На наступному етапі було реалізовано функцію для обчислення SHA 256 хешу. Вона приймає дані у вигляді байтів і повертає хеш у шістнадцятковому форматі.

```
def calculate_hash(data):
    return hashlib.sha256(data).hexdigest()


file_results = []

for filename in os.listdir("test_files"):
    filepath = f"test_files/{filename}"

    with open(filepath, "rb") as f:
        data = f.read()

    file_hash = calculate_hash(data)
    file_size = os.path.getsize(filepath)

    file_results.append({
        "Файл": filename,
        "Розмір файлу, байт": file_size,
        "SHA-256 хеш файлу": file_hash
    })

df_files = pd.DataFrame(file_results)
df_files
```

	Файл	Розмір файлу, байт	SHA-256 хеш файлу
0	file3.txt	705	37ddcd4ec054507ac0ae3e15d91b5b63c0afd3c2269539...
1	file2.txt	845	46cf654a98113351c92ccd8dbccd299c8484cdc233f367...
2	file4.txt	859	534922c6db650fa3bcedd8aed3e9d7d6d706c7e546685c...
3	file1.txt	859	534922c6db650fa3bcedd8aed3e9d7d6d706c7e546685c...

Рис.6 Обчислення хешів

Цей етап показує перехід від звичайної ідентифікації файлу за назвою до ідентифікації за вмістом. У звичайному підході файл визначається як file1.txt або file2.txt. В адресації за вмістом файл визначається через його хеш.

У результатах видно, що файли file1.txt і file4.txt мають однаковий SHA 256 хеш. Це означає, що вони мають однаковий вміст. Тобто з точки зору адресації за вмістом це один і той самий об’єкт, хоча назви файлів різні.

3.4. Розбиття файлів на блоки та створення сховища за хешем

Після обчислення хешів файлів було виконано розбиття файлів на блоки. Для цього задано розмір блоку 128 байт.

```
BLOCK_SIZE = 128

content_storage = {}
file_block_map = {}
block_rows = []
```

Рис. 7 Блоки

Кожен файл зчитується у вигляді байтів і ділиться на частини однакового розміру. Такий підхід імітує роботу розподілених сховищ, де великі файли часто зберігаються не як один об’єкт, а як набір блоків.

```
for filename in os.listdir("test_files"):
    filepath = f"test_files/{filename}"

    with open(filepath, "rb") as f:
        file_data = f.read()

    blocks = [
        file_data[i:i + BLOCK_SIZE]
        for i in range(0, len(file_data), BLOCK_SIZE)
    ]
```

Рис.8 Биття на блоки

Далі для кожного блоку обчислюється SHA 256 хеш. Цей хеш використовується як адреса блоку у сховищі content_storage.

```
    if block_hash not in content_storage:
        content_storage[block_hash] = block
        status = "Новий блок"
    else:
        status = "Дублікат"
```

Рис.9 Хеш для блоку

Це головний фрагмент практичної реалізації. Тут змодельовано сховище з адресацією за вмістом. В словнику content_storage ключем є хеш блоку, а значенням є сам блок даних. Тобто дані зберігаються не за назвою файлу і не за шляхом, а за хешем вмісту

Саме цей етап безпосередньо розкриває тему адресації за вмістом. У класичній IP моделі дані шукаються за місцем розташування. У цій моделі блоки шукаються і зберігаються за хешем. Тобто хеш виконує роль адреси.
Після виконання цього етапу було сформовано таблицю df_blocks. У ній для кожного блоку показано назву файлу, номер блоку, розмір блоку, хеш блоку та статус. Статус показує, чи є блок новим, чи він уже зустрічався раніше.

3.5. Результати роботи програми та пояснення отриманих даних

Після виконання всіх блоків програма сформувала кілька таблиць і два графіки. Спочатку було підраховано загальну кількість блоків, кількість унікальних блоків і кількість повторюваних блоків.

У результаті було отримано 27 блоків, з яких 18 є унікальними, а 9 є повторюваними. Це підтверджує, що частина даних дублюється і може не зберігатися повторно. У фінальних результатах також вказано, що загальна кількість блоків становить 27, кількість унікальних блоків 18, а кількість повторюваних блоків 9.

	Показник	Значення
0	Загальна кількість блоків	27
1	Кількість унікальних блоків	18
2	Кількість повторюваних блоків	9

Рис.10 Блоки

Далі було виконано розрахунок економії пам’яті. Початковий обсяг усіх файлів становив 3268 байт. Після дедуплікації обсяг унікальних блоків становив 2153 байт. Отже, було зекономлено 1115 байт, що складає 34.12%

	Показник	Значення
0	Початковий обсяг усіх файлів, байт	3268.00
1	Обсяг після дедуплікації, байт	2153.00
2	Зекономлено пам'яті, байт	1115.00
3	Економія пам'яті, %	34.12

Рис.11 Пам’ять

Цей результат показує практичну користь адресації за вмістом. Якщо однакові блоки мають однаковий SHA 256 хеш, їх не потрібно зберігати багато разів. Достатньо зберегти один унікальний блок і використовувати його повторно.

Окремо було виконано перевірку цілісності даних. Для цього було взято два майже однакові рядки:

	Стан даних	Дані	SHA-256 хеш
0	Початкові дані	Content addressing instead of IP	06c54166cce29112ea74b858eca9a7741995e7d145c334...
1	Дані після зміни одного символу	Content addressing instead of ip	7f7a8dc6fe02d46f35ae874cccb934b61962c047df2a53...

Рис.12 Вміст даних

В другому рядку змінено лише один символ. Після обчислення SHA 256 хешів було отримано різні значення. Це означає, що навіть мінімальна зміна вмісту призводить до зміни хешу. Такий результат підтверджує, що хешування можна використовувати для перевірки цілісності даних.

Перший графік показує співвідношення між усіма блоками, унікальними блоками та дублікатами. На ньому видно, що з 27 блоків тільки 18 потрібно зберігати як унікальні, а 9 блоків є повторюваними.

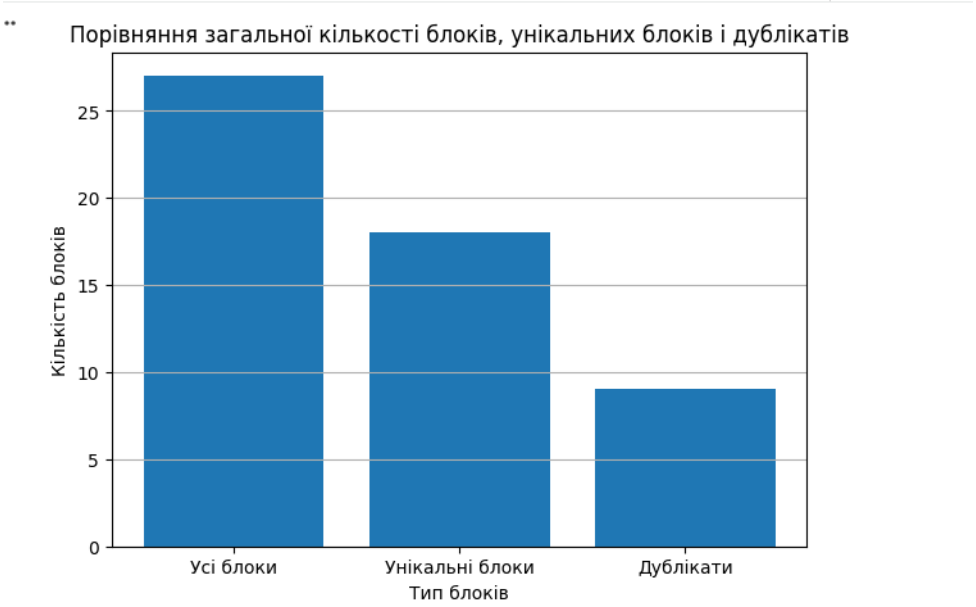


Рис.13 Графік співвідношень блоків

Другий графік показує економію пам’яті. На ньому порівнюється початковий обсяг даних, обсяг після дедуплікації та розмір зекономленої пам’яті. Графік наочно підтверджує, що після застосування адресації за вмістом і дедуплікації обсяг зберігання зменшується.



Рис.14 Графік економії памяті

Висновок.

У межах розрахунково графічної роботи було розглянуто тему «Принципи адресації за вмістом замість IP». У теоретичній частині було пояснено, що класична IP адресація орієнтована на місце розташування ресурсу, тобто на сервер або шлях до файлу, тоді як адресація за вмістом орієнтована на самі дані. Основою такого підходу є хешування, зокрема алгоритм SHA 256, який формує цифровий відбиток файлу або блоку даних. Завдяки цьому однаковий вміст має однаковий хеш, а будь яка зміна даних призводить до зміни хешу.

У практичній частині було реалізовано спрощену модель сховища з адресацією за вмістом. Файли були розбиті на блоки, для кожного блоку було обчислено SHA 256 хеш, після чого блоки зберігалися за цим хешем. У результаті було отримано 27 блоків, з яких 18 були унікальними, а 9 повторюваними. Початковий обсяг даних становив 3268 байт, після дедуплікації він зменшився до 2153 байт, а економія пам’яті склала 1115 байт або 34,12 відсотка. Також було показано, що зміна одного символу повністю змінює хеш, тому такий підхід можна використовувати для перевірки цілісності даних. Отже, практична частина підтвердила, що адресація за вмістом дозволяє ідентифікувати дані за їхнім хешем, знаходити дублікати, зменшувати обсяг зберігання і краще підходить для розподілених сховищ даних.

Додаток А.

Посилання на Google Colab:

https://colab.research.google.com/drive/1QixFZxFRkPzmTYIopjI8Kbs9Y6YwRuZZ?usp=sharing

Список використаних джерел.

1. RFC 6920. Naming Things with Hashes. Internet Engineering Task Force, 2013. URL: https://datatracker.ietf.org/doc/html/rfc6920
2. NIST. FIPS PUB 180-4: Secure Hash Standard (SHS). National Institute of Standards and Technology, 2015. URL: https://csrc.nist.gov/pubs/fips/180-4/upd1/final
3. IPFS Docs. Content Identifiers (CIDs). Офіційна документація IPFS. URL: https://docs.ipfs.tech/concepts/content-addressing/
4. IPFS Docs. Merkle Directed Acyclic Graphs (DAG). Офіційна документація IPFS. URL: https://docs.ipfs.tech/concepts/merkle-dag/ IPFS
5. Docs. Distributed Hash Tables (DHT). Офіційна документація IPFS. URL: https://docs.ipfs.tech/concepts/dht/
6. Python Documentation. hashlib Secure hashes and message digests. Офіційна документація Python. URL: https://docs.python.org/3/library/hashlib.html
7. Tanenbaum A. S., Wetherall D. J. Computer Networks. 5th ed. Pearson, 2011.
8. Coulouris G., Dollimore J., Kindberg T., Blair G. Distributed Systems: Concepts and Design. 5th ed. Addison Wesley, 2011.
9. Silberschatz A., Korth H. F., Sudarshan S. Database System Concepts. 7th ed. McGraw Hill, 2019.
10. Kleppmann M. Designing Data Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O’Reilly Media, 2017.